

← All Submissions

Accepted 61 / 61 testcases passed

youstab21 submitted at Jul 13, 2026 19:16

Analysis

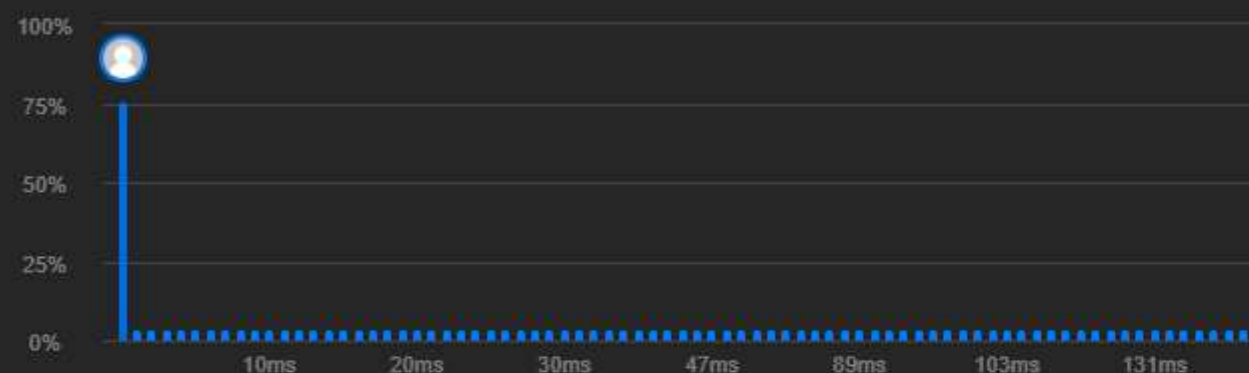
Solution

Runtime

0 ms | Beats 100.00%

Memory

20.69 MB | Beats 58.98%



Code | C++

```
1 class Solution {
2 public:
3     int singleNumber(vector<int>& nums) {
4         // here we can make use of the A XOR A = 0 to eliminate duplicates
5         // as there is an even number of duplicates of each element they all
6         // except the number that only appears once (odd number of times)
7         int ans = 0;
8         for(int i = 0; i < nums.size() ; i++){
```

View more

More challenges

260. Single Number III 389. Find the Difference

3158. Find the XOR of Numbers Which Appear Twice

Write your notes here

Select related tags

0/5

Code

C++ Auto

```
1 class Solution {
2 public:
3     int singleNumber(vector<int>& nums) {
4         // here we can make use of the A XOR A = 0 to eliminate duplicates
5         // as there is an even number of duplicates of each element they all xor to x
6         // except the number that only appears once (odd number of times)
7         int ans = 0;
8         for(int i = 0; i < nums.size() ; i++){
9             ans = ans ^ nums[i];
10        }
11        return ans;
12    }
13};
```

--INSERT--

Saved

Ln 13, Col 3

Testcase Test Result

[Description](#) | [Accepted](#) × | [Editorial](#) | [Solutions](#) | [Submissions](#)

[← All Submissions](#) [🔗](#)

Accepted 598 / 598 testcases passed

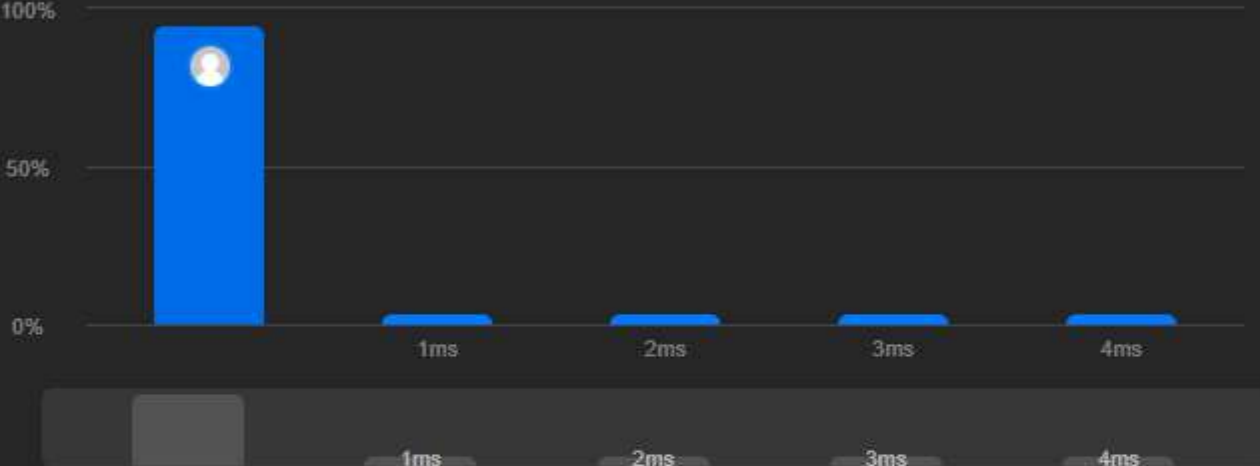
 Analysis [📄 Solution](#)

 Runtime

0 ms | Beats 100.00% 🏆

 Memory

8.09 MB | Beats 99.37% 🏆



Code | C++

```
1 class Solution {
2 public:
3     int hammingWeight(int n) {
4         int count = 0;
5         // loop through the 32 bits of the int
6         for (int i = 0 ;i < 32; i++){
7             // add the value of the first bit either 1 or 0 which we get by us
8             count = count + (n & 1);
9         }
10    }
11 }
```

 View more

More challenges

• [231. Power of Two](#)

• [338. Counting Bits](#)

• [762. Prime Number of Set Bits in Binary Representation](#)

Write your notes here

Select related tags 0/5

</> Code

C++ ▾ 🔒 Auto

```
1 class Solution {
2 public:
3     int hammingWeight(int n) {
4         int count = 0;
5         // loop through the 32 bits of the int
6         for (int i = 0 ;i < 32; i++){
7             // add the value of the first bit either 1 or 0 which we get by using bitwise and
8             count = count + (n & 1);
9             // shift the number right to check the next bit in the next iteration
10            n = n >> 1;
11        }
12        return count;
13    }
14 };
```

--INSERT--

Saved

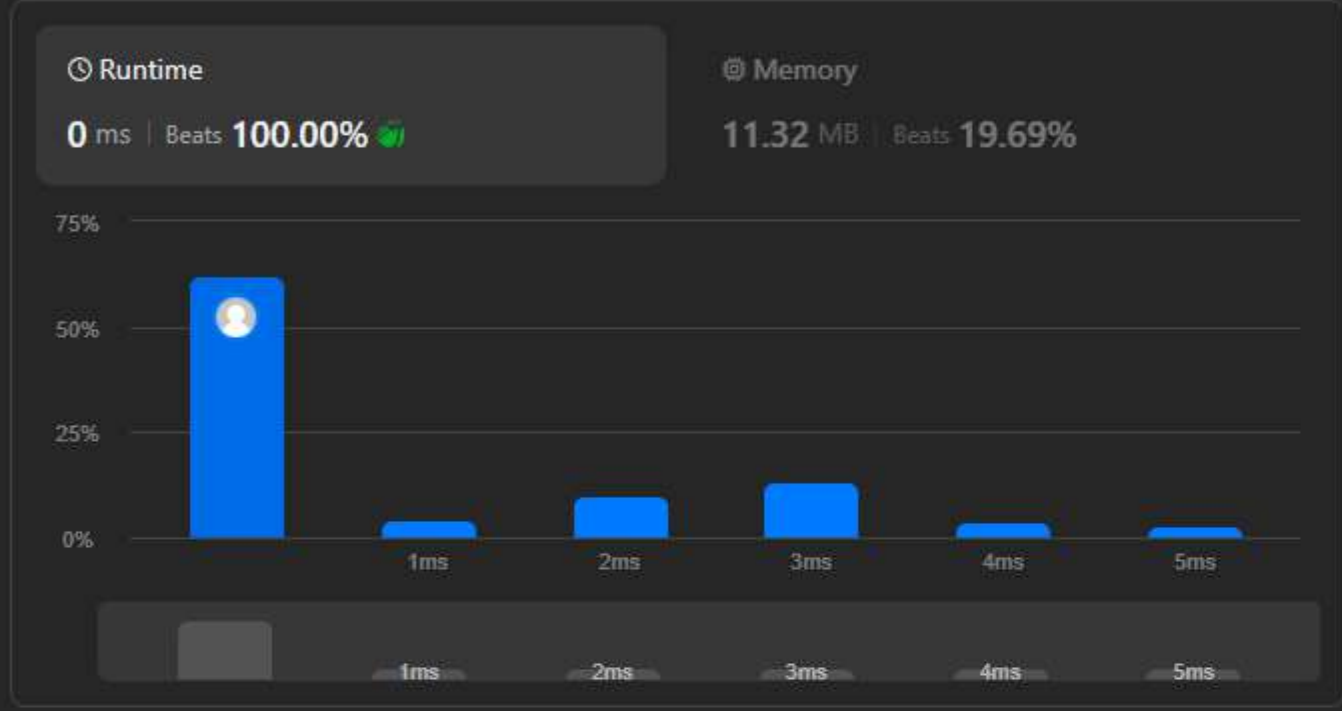
Ln 12, Col 22

 Testcase | [> Test Result](#)

[Description](#) | [Accepted](#) x | [Editorial](#) | [Solutions](#) | [Submissions](#)

← All Submissions [🔗](#)

Accepted 15 / 15 testcases passed [🔍 Analysis](#) [📄 Solution](#)
👤 **yousab21** submitted at Jul 13, 2026 20:36



Code | C++

```
1 // i had to look up the O(n) solution and iam glad i did as it way moer intr
2 // the idea is that if you shift a number to the right the number of set bit
3 // was set to one, but shifting a number to the right is also the same as de
4 // that means you can reuse calculations from smaller numbers to calculate l
5 class Solution {
6 public:
7     vector<int> countBits(int n) {
8         vector<int> setBits;
```

[👁 View more](#)

More challenges

[• 2859. Sum of Values at Indices With K Set Bits](#)

[• 2917. Find the K-or of an Array](#)

Write your notes here

Select related tags 0/5

Code C++ v Auto [☰](#) [🔖](#) [{ }](#) [↶](#) [↷](#)

```
1 // i had to look up the O(n) solution and iam glad i did as it way moer intresting
2 // the idea is that if you shift a number to the right the number of set bits only change if the rightmost bit
3 // was set to one, but shifting a number to the right is also the same as deviding by 2
4 // that means you can reuse calculations from smaller numbers to calculate larger ones
5 class Solution {
6 public:
7     vector<int> countBits(int n) {
8         vector<int> setBits;
9         // sets the number of set bits 0 for binary represnted 0
10        setBits.push_back(0);
11
12        // now the intresting part
13        int bitCount = 0;
14        for(int i = 1 ; i < n + 1 ; i++){
15            // shifting the current index by 1 is the same as deviding by 2
16            // but also the shifted number has the same number of ones - whatever was in the bit lost by shifting
17            // so we just re-add whatever value was said bit
18            // you end up with an operation like { setBits[11] = setBits[5] + 1 }
19            // in binary : onesIn( 1101 ) = onesIn( 110 ) + 1
20            bitCount = setBits[i >> 1] + (i & 1);
21            setBits.push_back(bitCount);
22        }
23        return setBits;
24    }
25};
```

--INSERT--

Saved Ln 22, Col 10

[📄 Testcase](#) | [➤ Test Result](#)

Description

Accepted

Editorial

Solutions

Submissions

All Submissions

Accepted

122 / 122 testcases passed

Analysis

Solution

yousab21

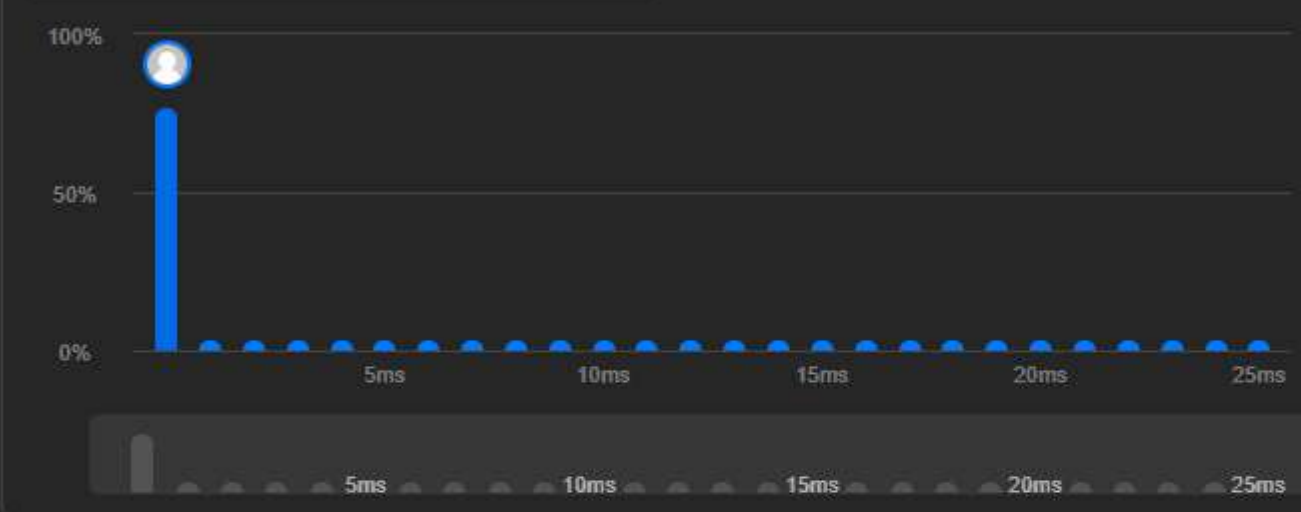
submitted at Jul 14, 2026 14:43

Runtime

0 ms | Beats 100.00%

Memory

21.74 MB | Beats 70.86%



Code

C++

```
1 // i couldnt find the bitwise solution but i found another one in O(N)
2 // the idea is scine the numbers are all positive and continuos they should
3 // to the standard fourmela for a sum of n integers " (n-1) * n / 2 " so if
4 // the actual numbers of the array and substract from the value of the fourm
5 class Solution {
6 public:
7     int missingNumber(vector<int>& nums) {
8         // notice here nums.size() + 1 because the range we are summing is l
```

View more

More challenges

41. First Missing Positive

765. Couples Holding Hands

3471. Find the Largest Almost Missing Integer

Write your notes here

Select related tags

0/5

Code

C++

Auto

```
1 // i couldnt find the bitwise solution but i found another one in O(N)
2 // the idea is scine the numbers are all positive and continuos they should
3 // to the standard fourmela for a sum of n integers " (n-1) * n / 2 " so if we sum the
4 // the actual numbers of the array and substract from the value of the fourmela we get the missign number
5 class Solution {
6 public:
7     int missingNumber(vector<int>& nums) {
8         // notice here nums.size() + 1 because the range we are summing is larger than the one in
9         // the array by one because of the missing number
10        int fullRangeSum = (nums.size() * (nums.size() + 1)) / 2;
11        int actualSum = 0;
12
13        for(int i = 0; i < nums.size(); i++){
14            actualSum += nums[i];
15        }
16
17        return fullRangeSum - actualSum;
18    }
19};
```

--NORMAL--

Saved

Ln 1, Col 1

Testcase

Test Result

Description Accepted X Editorial Solutions Submissions

All Submissions

Accepted 31 / 31 testcases passed

yousab21 submitted at Jul 14, 2026 14:56

Analysis

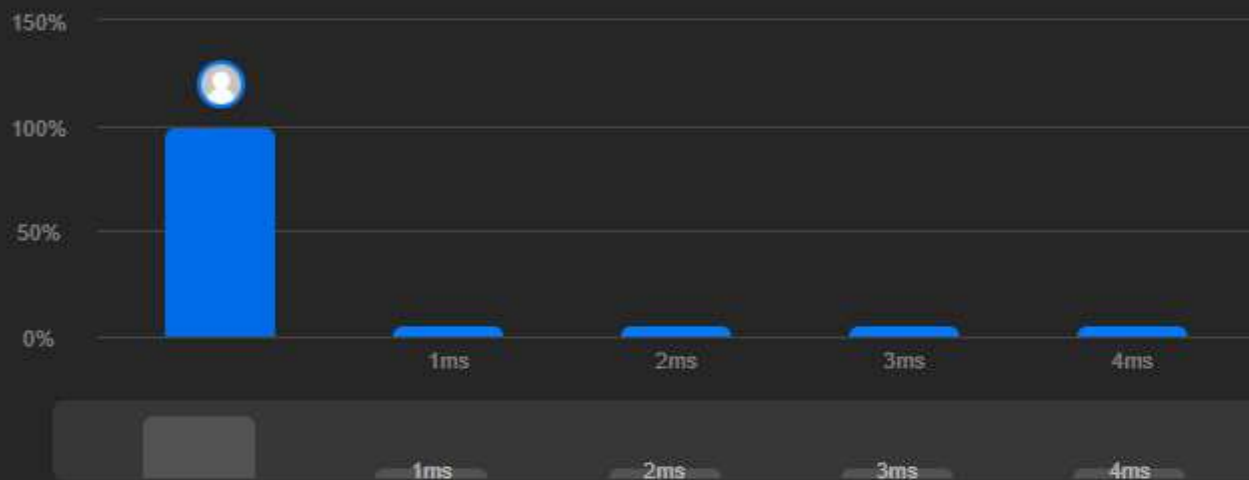
Solution

Runtime

0 ms | Beats 100.00%

Memory

7.87 MB | Beats 35.89%



Code C++

```
1 // i think the smart solution here is to basicly implement the parallel adder
2 // using some type of recurssive function
3 // however debugging bitwise code is hell so i dont want to do that, insted
4 // i came up with this O(n) solution using increment and decrement operators
5 class Solution {
6 public:
7     int getSum(int a, int b) {
8         // if b is positive just increment a and decrement b until b is 0
```

View more

More challenges

1281. Subtract the Product and Sum of Digits of an Integer

1414. Find the Minimum Number of Fibonacci Numbers Whose Sum Is K

2949. Count Beautiful Substrings II

Write your notes here

Code

Submit Ctrl Enter

C++ Auto

```
1 // i think the smart solution here is to basicly implement the parallel adder circuit from logic 1
2 // using some type of recurssive function
3 // however debugging bitwise code is hell so i dont want to do that, insted
4 // i came up with this O(n) solution using increment and decrement operators
5 class Solution {
6 public:
7     int getSum(int a, int b) {
8         // if b is positive just increment a and decrement b until b is 0
9         while(b > 0){
10             a++;
11             b--;
12         }
13         // if b is negative just decrement a and increment b untile b is 0
14         while(b < 0){
15             a--;
16             b++;
17         }
18         return a;
19     }
20 }
21 ;;
```

--INSERT--

Saved

Ln 13, Col 61

Testcase Test Result

DescriptionAccepted x Editorial SolutionsSubmissions

All Submissions

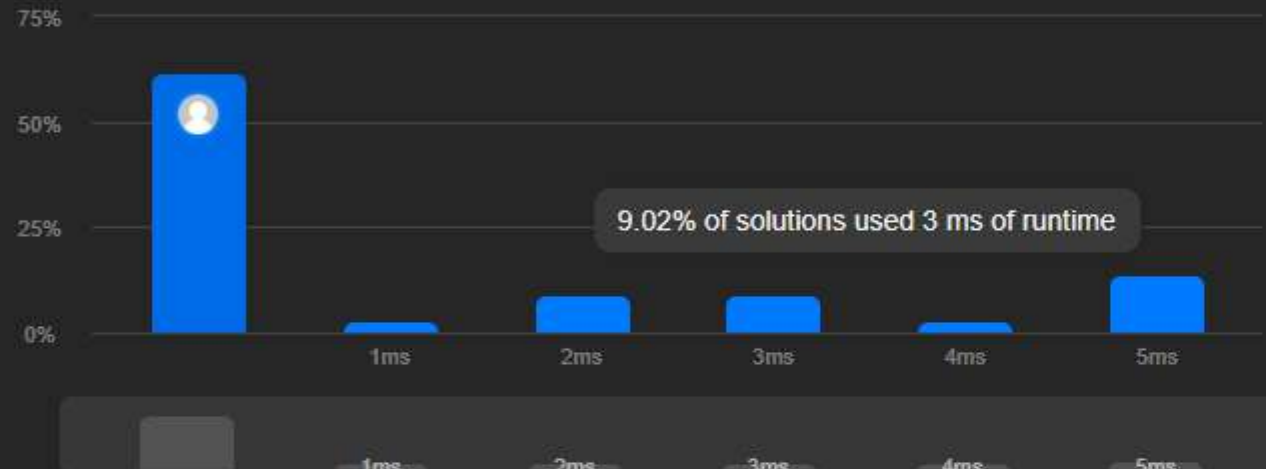
Accepted1045 / 1045 testcases passed

yousab21 submitted at Jul 14, 2026 21:22

AnalysisSolution

Runtime0 ms | Beats 100.00%

Memory8.44 MB | Beats 83.89%



Code | C++

```
1 // the code itself is actually really simple it is only that long because the test cases are filled
2 // with integer overflow cases
3 // also the hardest part is the problem's challenge to not use long long as
4 // this forces you to check for overflows before they happen
5 class Solution {
6     // this does what is says in the name, the while condition just for an
7     void removeTrailingZeros(int& x){
8         while(x != 0 && x % 10 == 0){
9             x = x / 10;
10        }
11    }
12
13    // this is the exact fast integer power method we learnt in the week material
14    // just modified to check for int overflow before doing any multiplication
15    // if an int overflow is about to happen the function just returns -1
16    int fastPower(int base, int power){
17        int result = 1;
18        while(power > 0){
19            if(power % 2 == 1){
20                result = result * base;
21            }
22
23            power = power / 2;
24
25            if (power > 0){
26                if (base > INT_MAX / base){
27                    return -1;
28                }
29                base = base * base;
30            }
31        }
32        return result;
33    }
34
35    // this just counts the number of digits in the number i will explain why i do this later
36    int numOfDigits(int x){
37        int count = 0;
38        while (x > 0){
39            x = x / 10;
40            count ++;
41        }
42        return count;
43    }
44}
```

More challenges

8. String to Integer (atoi)

2119. A Number After a Double Reversal

2442. Count Number of Distinct Integers After Reverse Operations

Write your notes here

Select related tags0/5

CodeC++ v Auto

```
1 // the code itself is actually really simple it is only that long because the test cases are filled
2 // with integer overflow cases
3 // also the hardest part is the problem's challenge to not use long long as
4 // this forces you to check for overflows before they happen
5 class Solution {
6     // this does what is says in the name, the while condition just for an the edgecase where x = 0
7     void removeTrailingZeros(int& x){
8         while(x != 0 && x % 10 == 0){
9             x = x / 10;
10        }
11    }
12
13    // this is the exact fast integer power method we learnt in the week material
14    // just modified to check for int overflow before doing any multiplication
15    // if an int overflow is about to happen the function just returns -1
16    int fastPower(int base, int power){
17        int result = 1;
18        while(power > 0){
19            if(power % 2 == 1){
20                result = result * base;
21            }
22
23            power = power / 2;
24
25            if (power > 0){
26                if (base > INT_MAX / base){
27                    return -1;
28                }
29                base = base * base;
30            }
31        }
32        return result;
33    }
34
35    // this just counts the number of digits in the number i will explain why i do this later
36    int numOfDigits(int x){
37        int count = 0;
38        while (x > 0){
39            x = x / 10;
40            count ++;
41        }
42        return count;
43    }
44}
```

--NORMAL--??(JavaScript regexp)

SavedLn 1, Col 1

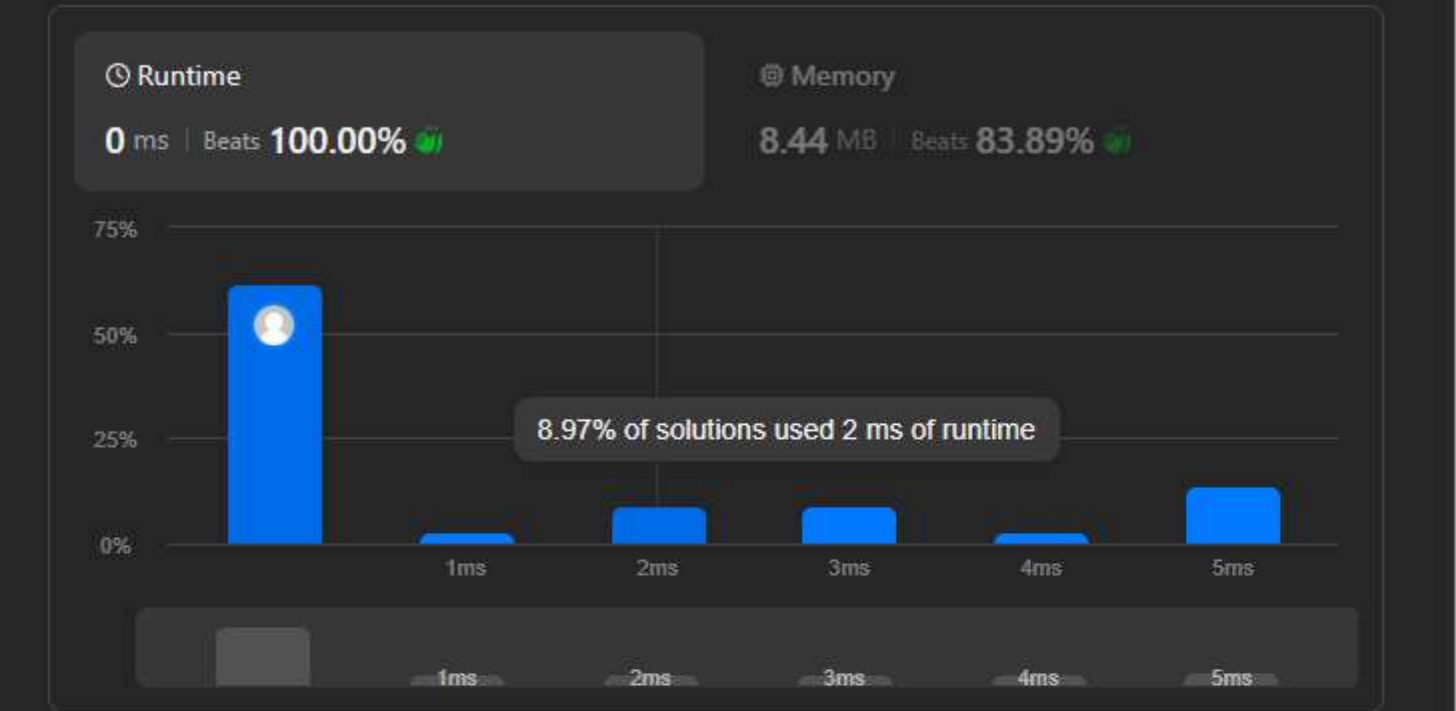
TestcaseTest Result

Description Accepted x Editorial Solutions Submissions

All Submissions

Accepted 1045 / 1045 testcases passed
 yousab21 submitted at Jul 14, 2026 21:22

Analysis Solution



Code | C++

```
1 // the code itself is actually really simple it is onlt that long becuae th
2 // with integer overflow casses
3 // also the hardest part is the problem's challage to not use long long as
4 // this forces you to check for overflows before they happen
5 class Solution {
6     // this deos what is says in the name, the while condition just for an
7     void removeTrailingZeros(int& x){
8         while(x != 0 && x % 10 == 0){
```

More challenges

- 8. String to Integer (atoi)
- 2119. A Number After a Double Reversal
- 2442. Count Number of Distinct Integers After Reverse Operations

Write your notes here

Select related tags 0/5

Code

C++ Auto

```
41 }
42 return count;
43 }
44
45 public:
46 int reverse(int x) {
47     // to massivly simplyfy calculations and overflow checks i treat every number as a positive
48     // and then just re-add the sign before i return if it was negative using a flag i set
49     bool isNegative = false;
50     if (x < 0){
51         // this is intresting as the minimumm int is 2^-31 but the largest int is 2^31 -1
52         // so the minimumm int cant be multiplies by -1 as it will just 1 over the max value
53         if(x == INT_MIN){
54             return 0;
55         }
56         isNegative = true;
57         x = x * -1;
58     }
59
60     //trailing zeros dont have a meaning in reverse so i just remove them from the start
61     removeTrailingZeros(x);
62
63     int digit = 0;
64     int digitValue = 0;
65     int placeValue = 0;
66     int reversed = 0;
67     // i build the number in reverse from left to right from the highest power of 10 to the lowest
68     // this why i need to count the digits and the highest power of 10 is just the number of digits - 1
69     int power = numOfDigits(x) - 1;
70
71     while(x > 0){
72         // i begin by extracting the right most digit
73         digit = x % 10;
74
75         // we then need to try to calculate the value of the digit in its intended place
76         // to avoid over flows we need to do this in multiple steps
77
78         // if the fastPower function itseld said no then we exit
79         placeValue = fastPower(10, power);
80         if (placeValue == -1){
81             return 0;
82         }
83
84         // even if we were able to compute the power of 10 but we cant multioly it by the digit value
```

--NORMAL--?? (JavaScript regexp)

Saved

Testcase Test Result

Description

Accepted

Editorial

Solutions

Submissions

All Submissions

🔗

Accepted

1045 / 1045 testcases passed

🔍 Analysis

📄 Solution

👤 yousab21

submitted at Jul 14, 2026 21:22

🕒 Runtime

0 ms | Beats 100.00%

🧠 Memory

8.44 MB | Beats 83.89%

Runtime	Beats
0 ms	100.00%
1 ms	0.00%
2 ms	0.00%
3 ms	0.00%
4 ms	0.00%
5 ms	0.00%

Code

C++

```
1 // the code itself is actually really simple it is only that long because th
2 // with integer overflow cases
3 // also the hardest part is the problem's challenge to not use long long as
4 // this forces you to check for overflows before they happen
5 class Solution {
6     // this does what is says in the name, the while condition just for an
7     void removeTrailingZeros(int& x){
8         while(x != 0 && x % 10 == 0){
9
10         }
11     }
12 }
```

👁 View more

More challenges

• 8. String to Integer (atoi)

• 2119. A Number After a Double Reversal

• 2442. Count Number of Distinct Integers After Reverse Operations

Write your notes here

Select related tags

0/5

Code

C++

Auto

```
75 // we then need to try to calculate the value of the digit in its intended place
76 // to avoid over flows we need to do this in multiple steps
77
78 // if the fastPower function itself said no then we exit
79 placeValue = fastPower(10, power);
80 if (placeValue == -1){
81     return 0;
82 }
83
84 // even if we were able to compute the power of 10 but we cant multiply it by the digit value
85 // we also exit
86 if (digit != 0 && placeValue > INT_MAX / digit){
87
88     return 0;
89 }
90
91 // only then that we can actually compute the digit value
92 digitValue = digit * placeValue;
93
94 // we now also need to add that digit value to the reversed number
95 // we just check first if the addition will overflow
96 if (reversed > INT_MAX - digitValue){
97     return 0;
98 }
99
100 // if it will not we can finally add the digit value to the number
101 reversed = reversed + digitValue;
102
103 // then to advance to the next digit we decrease the power to be to the right of the digit we just wrote
104 power--;
105
106 // and we divide by ten to prepare to fetch the next digit
107 x = x / 10;
108 }
109
110 // re-apply the negative flag
111 if(isNegative){
112     reversed = reversed * -1;
113 }
114
115 return reversed;
116 }
117 };
```

--NORMAL--??

(JavaScript regexp)

Saved

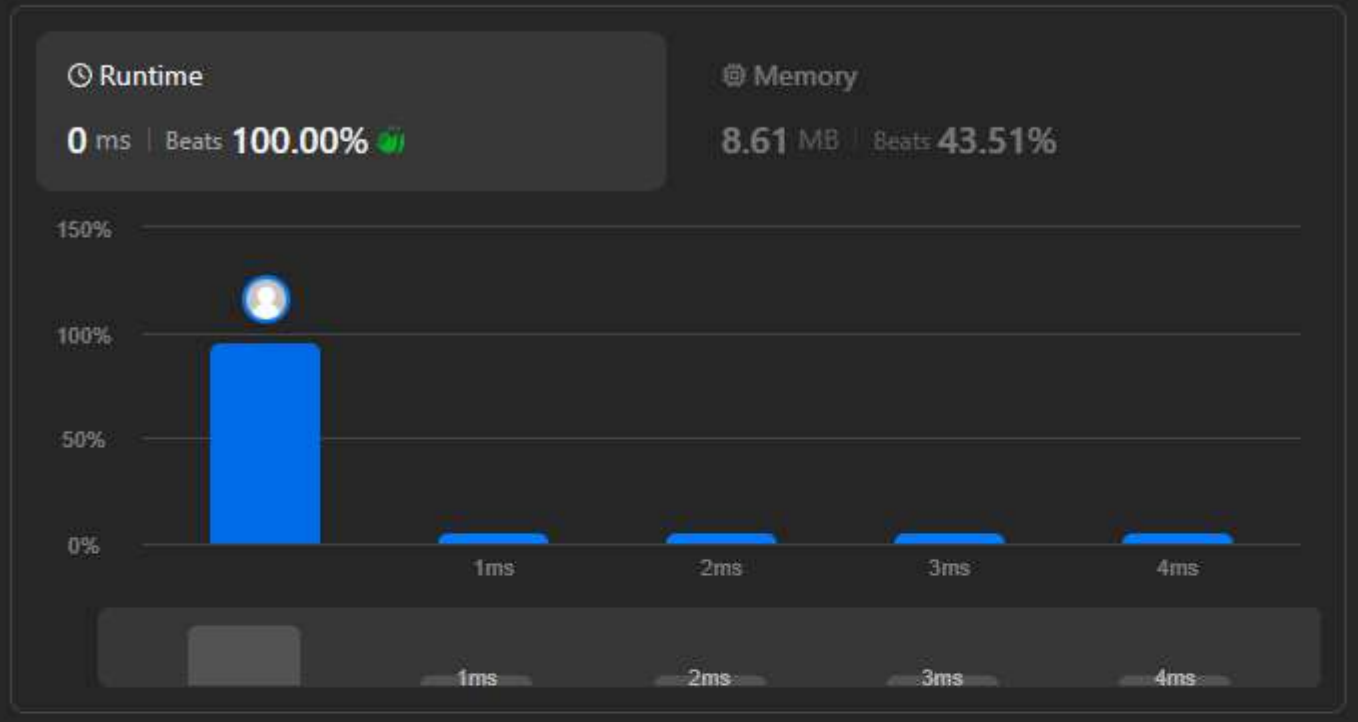
Ln 1, Col 1

🔍 Testcase

📄 Test Result

Accepted 307 / 307 testcases passed
yousab21 submitted at Jul 14, 2026 22:25

Analysis Solution



Code | C++

```
1 class Solution {
2 public:
3     double myPow(double x, int n) {
4         // this time the problem said nothing about the not using long long
5         // so i just convert the power to a long long to negate the edge case
6         // where -2^-31 cant fit in a positive int
7         long long N = n;
8     }
```

- More challenges
- 69. Sqrt(x)
 - 372. Super Pow
 - 2550. Count Collisions of Monkeys on a Polygon

Write your notes here

Select related tags 0/5

```
1 class Solution {
2 public:
3     double myPow(double x, int n) {
4         // this time the problem said nothing about the not using long long
5         // so i just convert the power to a long long to negate the edge case
6         // where -2^-31 cant fit in a positive int
7         long long N = n;
8
9         // insted of writig the logic twice for powers and roots
10        // if n is negative i just take flix x to 1/x so that multipling by
11        // it is the same as deviding over x
12        if (N < 0) {
13            x = 1 / x;
14            N = N * -1;
15        }
16        // this is just thestandard fast power algorithem nagetive powers
17        // just get handeled by flipping x in the previose step
18        double result = 1;
19        while (N > 0) {
20            if (N % 2 == 1) {
21                result = result * x;
22            }
23            x = x * x;
24            N = N / 2;
25        }
26        return result;
27    }
28 };
```


Description Accepted x Editorial Solutions Submissions

All Submissions

Accepted 296 / 296 testcases passed

yousab21 submitted at Jul 15, 2026 17:40

Analysis Solution

Runtime

0 ms | Beats 100.00%

Memory

12.30 MB | Beats 5.28%

Runtime	1ms	2ms	3ms	4ms
Beats	100.00%	~0.5%	~0.5%	~0.5%

Code | C++

```
1 class Solution {
2     // i litirally just implemented the full addder circuit from logic 1
3     int getSum(int carryIn, int x, int y){
4         return carryIn ^ x ^ y;
5     }
6     int getCarryOut(int carryIn, int x, int y){
7         return ( (x & y) | (x & carryIn) | (y & carryIn) );
8     }
9 }
```

View more

More challenges

43. Multiply Strings 989. Add to Array-Form of Integer

Write your notes here

Select related tags 0/5

👍 🗜️ 🌐 🌱 🌿 🍀

Code

C++ Auto

```
1 class Solution {
2     // i litirally just implemented the full addder circuit from logic 1
3     int getSum(int carryIn, int x, int y){
4         return carryIn ^ x ^ y;
5     }
6     int getCarryOut(int carryIn, int x, int y){
7         return ( (x & y) | (x & carryIn) | (y & carryIn) );
8     }
9 public:
10    string addBinary(string a, string b) {
11        // we start with the lease signigicant bits (right of the string)
12        int i = a.size() - 1;
13        int j = b.size() - 1;
14
15        int carry = 0;
16        string result = "";
17
18        // run until both strings are done and there is no leftover carry
19        while(i ≥ 0 || j ≥ 0 || carry){
20            // thia expretion just gets the value of the bit as an int until it the string
21            // runs out then it returns 0, { char - '0'} substracts the ascii code for 0 so
22            // it covernts a numirical char to the actual int
23            int x = (i ≥ 0) ? a[i] - '0' : 0;
24            int y = (j ≥ 0) ? b[j] - '0' : 0;
25
26            int sum = getSum(carry, x, y);
27            carry = getCarryOut(carry, x, y);
28
29            // order i concat the string here really matters as addition has to be from
30            // least segnificant bit to mosr significant bit to make sense
31            result = to_string(sum) + result;
32
33            // move pointers left
34            i--; j--;
35        }
36
37        return result;
38    }
39 };
```

--INSERT--

Saved

Ln 18, Col 64

Testcase Test Result

 **Solution**

≡ View more

- 43. Multiply Strings
- 369. Plus One Linked List
- 989. Add to Array-Form of Integer

0/5

C++ Auto

Testcase | **Test Result**

Ln 27, Col 1

Accepted 420 / 420 testcases passed
yousab21 submitted at Jul 16, 2026 16:27

Analysis Solution

Runtime

0 ms | Beats 100.00%

Memory

7.83 MB | Beats 68.76%



Code | C++

```
1 // the whole "loops endlessly" ringed a bell for me from the two pointers we
2 // here is how it works :
3 class Solution {
4     // this function gets the next sum for a passed number
5     // this acts like the "next" pointer in a linked list fot 2 pointers log
6     int nextSum(int n){
7         int sum = 0;
8     }
```

View more

More challenges

- 258. Add Digits
- 263. Ugly Number
- 2457. Minimum Addition to Make Integer Beautiful

Write your notes here

Select related tags

0/5

```
1 // the whole "loops endlessly" ringed a bell for me from the two pointers week
2 // here is how it works :
3 class Solution {
4     // this function gets the next sum for a passed number
5     // this acts like the "next" pointer in a linked list fot 2 pointers logic
6     int nextSum(int n){
7         int sum = 0;
8
9         // i just get each digit and sum its sqaure digit by digit then advance digits
10        // by devising by 10
11        while(n){
12            sum += (n % 10) * (n % 10);
13            n = n/ 10;
14        }
15
16        return sum;
17    }
18 public:
19     bool isHappy(int n) {
20         // i define both fast and slow pointers to the original stasrting number
21         int fast = n , slow = n;
22         // here it is important to do a do while as doing a while loop exsits
23         // imediatly as both pointers start from the same number
24         do{
25             // fast and slow bread and butter
26             fast = nextSum(nextSum(fast));
27             slow = nextSum(slow);
28
29             // here is the actaul check for a happy number
30             // if the number is not happy they just loop until fast catches slow
31             // this detecting a loop we then exsit and return false
32             if(fast == 1 || slow == 1 ){
33                 return true;
34             }
35         }
36         while(fast != slow);
37
38         return false;
39     }
40 };
```

--INSERT--

Saved

Ln 31, Col 68

Testcase Test Result

Accepted 127 / 127 testcases passed
yousab21 submitted at Jul 16, 2026 16:58

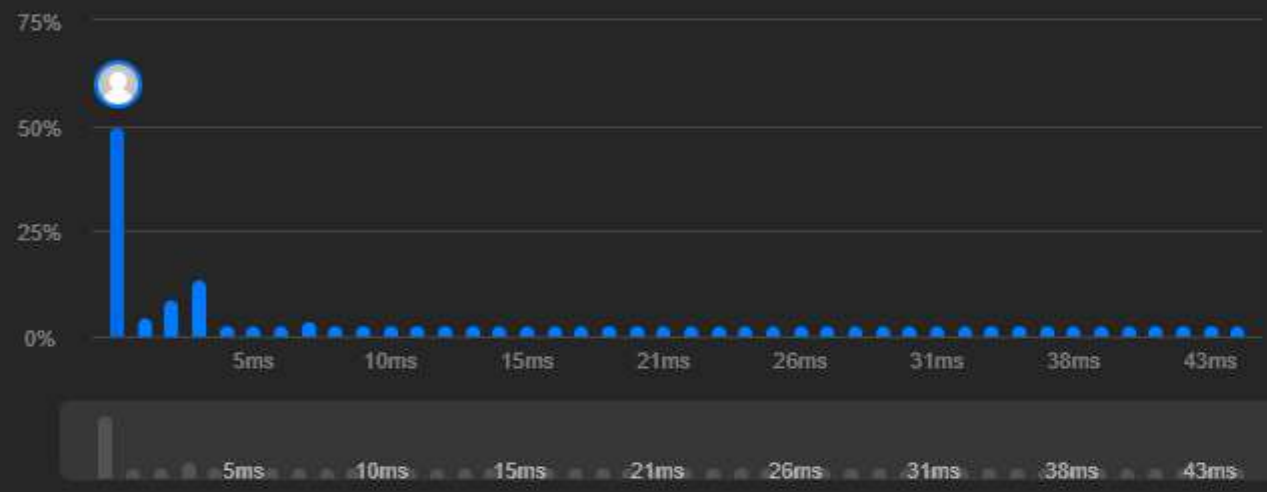
Analysis Solution

Runtime

0 ms | Beats 100.00%

Memory

7.72 MB | Beats 55.47%



Code C++

```
1 // my original solution was about binary seraching the whole possitive integ
2 // i looked up the best solution and it is actually more intresting here is
3 class Solution {
4 public:
5     bool judgeSquareSum(int c) {
6         // it is litirally left and right pointers fitted for the problem
7         //the left pointer starts from 0 and the right pointer starts from t
8         // as the smallest the left pointer will get is 0 so the laregst res
```

View more

More challenges

367. Valid Perfect Square

2778. Sum of Squares of Special Elements

Write your notes here

Select related tags

0/5



Code

C++ Auto

```
1 // my original solution was about binary seraching the whole possitive integer value space from 0 to 2^31
2 // i looked up the best solution and it is actually more intresting here is how it works
3 class Solution {
4 public:
5     bool judgeSquareSum(int c) {
6         // it is litirally left and right pointers fitted for the problem
7         //the left pointer starts from 0 and the right pointer starts from the sqrt of the number
8         // as the smallest the left pointer will get is 0 so the laregst resonable right should be the sqrt of c
9         // these numbers have to be long long even if they will never overflow
10        // because if there were int the sum variable would be cast to an int and that CAN overflow
11        long long a = 0;
12        long long b = sqrt(c);
13        while(a <= b){
14            // i check for the condiion in exacly the same way left and right pointers work
15            long long sum = a * a + b * b;
16            if(sum == c){
17                return true;
18            }
19
20            else if(sum > c){
21                b--;
22            }
23
24            else{
25                a++;
26            }
27        }
28        return false;
29    }
30};
```

--INSERT--

Saved

Ln 27, Col 10

Testcase Test Result